C++20 Concepts & Constraints

Theory, Edge Cases & Code Examples

- ✓ The Problem They Solve
- ✓ Concept Syntax and Declaration
- ✓ Standard Concepts Library (``)
- ✓ Requires Expressions and Clauses
- ✓ Constrained Auto and Abbreviated Function Templates

→ Swipe to learn →

C++20 Concepts & Constraints

Theory

📌 1. What Are Concepts - The Problem They Solve (1/2)

The Pre-C++20 Problem:

```
// ❌ C++17: No constraints - accepts ANY type
template<typename T>
T add(T a, T b) {
    return a + b;
}

// Compiles:
add(5, 10);           // ✅ OK: int
add(3.14, 2.71);      // ✅ OK: double

// Also compiles (but shouldn't!):
add(std::string("hello"), std::string("world"));  // ✅ Compiles! (has operator+)
add(std::vector<int>{}, std::vector<int>{});      // ❌ Error (but deep in template instantiation)

// Error message is PAGES long and mentions internal std::vector details!
```

The C++20 Solution with Concepts:

```
// ✅ C++20: Explicit constraints
template<typename T>
concept Arithmetic = std::is_arithmetic_v<T>;

template<Arithmetic T>
T add(T a, T b) {
    return a + b;
}

// Now:
add(5, 10);           // ✅ OK: int is arithmetic
add(3.14, 2.71);      // ✅ OK: double is arithmetic
add(std::string("hello"), std::string("world"));  // ❌ Clear error: "std::string does not satisfy Arithmetic"
```

C++20 Concepts & Constraints

Theory

📌 1. What Are Concepts - The Problem They Solve (2/2)

Benefits Table:

Aspect	Pre-C++20 (SFINAE)	C++20 Concepts
Readability	`std::enable_if_t<std::is_arithmetic_v<T>>`	`template<Arithmetic T>`
Error messages	100+ lines of template instantiation errors	"constraint not satisfied"
Composability	Complex nested conditions	`concept1 && concept2`
Overload resolution	SFINAE tricks required	Natural subsumption rules
Documentation	Comments only	Self-documenting constraints
Compile time	Slower (template instantiation)	Faster (early rejection)

C++20 Concepts & Constraints

Theory

📌 2. Concept Syntax and Declaration (1/2)

Basic Concept Declaration:

```
// Syntax: concept ConceptName = boolean-expression;
template<typename T>
concept Integral = std::is_integral_v<T>;

template<typename T>
concept SignedIntegral = Integral<T> && std::is_signed_v<T>;

template<typename T>
concept FloatingPoint = std::is_floating_point_v<T>;
```

Four Ways to Use Concepts:

```
// 1. Requires clause after template parameters
template<typename T>
    requires std::integral<T>
void func1(T x) {}

// 2. Trailing requires clause
template<typename T>
void func2(T x) requires std::integral<T> {}

// 3. Constrained template parameter (most common)
template<std::integral T>
void func3(T x) {}

// 4. Abbreviated function template (constrained auto)
void func4(std::integral auto x) {}

// All four are IDENTICAL in functionality!
```

C++20 Concepts & Constraints

Theory

📌 2. Concept Syntax and Declaration (2/2)

Syntax Comparison Table:

Style	Example	When to Use
Requires clause	`template<typename T> requires C<T>`	Multiple constraints, complex logic
Trailing requires	`void f(T x) requires C<T>`	When constraint depends on deduced types
Constrained parameter	`template<C T>`	**Most readable, preferred**
Abbreviated template	`void f(C auto x)`	Short functions, lambdas

Concept with Multiple Parameters:

```
template<typename T, typename U>
concept Comparable = requires(T a, U b) {
    { a == b } -> std::convertible_to<bool>;
    { a != b } -> std::convertible_to<bool>;
    { a < b } -> std::convertible_to<bool>;
};

// Usage:
template<typename T, typename U>
    requires Comparable<T, U>
bool less_than(T a, U b) {
    return a < b;
}
```

C++20 Concepts & Constraints

Theory

📌 3. Standard Concepts Library (``) (1/3)

C++20 provides **20+ standard concepts** organized by category:

Core Language Concepts:

Concept	Checks	Example
`std::same_as<T, U>`	T and U are the same type	`same_as<int, int>` ✅
`std::derived_from<T, U>`	T derives from U	`derived_from<Dog, Animal>` ✅
`std::convertible_to<T, U>`	T can convert to U	`convertible_to<int, double>` ✅
`std::common_reference_with<T, U>`	T and U have common reference type	
`std::common_with<T, U>`	T and U have common type	
`std::integral<T>`	T is an integral type	`integral<int>` ✅
`std::signed_integral<T>`	T is signed integral	`signed_integral<int>` ✅
`std::unsigned_integral<T>`	T is unsigned integral	`unsigned_integral<size_t>` ✅
`std::floating_point<T>`	T is floating-point	`floating_point<double>` ✅

Comparison Concepts:

Concept	Requires	Example
`std::equality_comparable<T>`	`a == b`, `a != b` return bool	Most types
`std::totally_ordered<T>`	`a < b`, `a <= b`, `a > b`, `a >= b`	Ordered types
`std::three_way_comparable<T>`	`a <=> b` valid	C++20 types with spaceship

C++20 Concepts & Constraints

Theory

📌 3. Standard Concepts Library (``) (2/3)

Object Concepts:

Concept	Checks	Notes
`std::movable<T>`	Can be moved	Move constructor + move assignment
`std::copyable<T>`	Can be copied	Copy constructor + copy assignment + movable
`std::semiregular<T>`	Copyable + default constructible	Like built-in types
`std::regular<T>`	Semiregular + equality comparable	Fully regular type

Callable Concepts:

Concept	Checks	Example
`std::invocable<F, Args...>`	Can call `f(args...)`	Any callable
`std::predicate<F, Args...>`	Invocable returning bool	Predicate functions
`std::relation<F, T, U>`	Binary predicate for relation	Comparison predicates

C++20 Concepts & Constraints

Theory

📌 3. Standard Concepts Library (``) (3/3)

Example Usage:

```
// Require any numeric type
template<typename T>
    requires std::integral<T> || std::floating_point<T>
T square(T x) {
    return x * x;
}

// Require copyable and comparable
template<std::copyable T>
    requires std::equality_comparable<T>
std::vector<T> remove_duplicates(std::vector<T> vec) {
    // Implementation...
}

// Require regular type (acts like built-in type)
template<std::regular T>
class Optional {
    // Can store any regular type
};
```

C++20 Concepts & Constraints

Theory

📌 4. Requires Expressions and Clauses (1/3)

Requires Expression Syntax:

A `requires` expression checks if a set of requirements are satisfied:

```
template<typename T>
concept HasPushBack = requires(T container, typename T::value_type value) {
    container.push_back(value);              // Simple requirement
    { container.size() } -> std::same_as<std::size_t>;  // Type requirement
    typename T::iterator;                    // Type alias requirement
    requires std::same_as<typename T::value_type, int>;  // Nested requirement
};
```

Four Types of Requirements:

1. Simple Requirements - Check expression validity:

```
template<typename T>
concept Incrementable = requires(T x) {
    ++x;   // Just checks if ++x is valid
    x++;   // Just checks if x++ is valid
};
```

2. Type Requirements - Check type validity:

```
template<typename T>
concept Container = requires {
    typename T::value_type;     // Must have value_type
    typename T::iterator;        // Must have iterator
    typename T::const_iterator;  // Must have const_iterator
};
```

3. Compound Requirements - Check expression + return type:

```
template<typename T>
concept Range = requires(T range) {
    { range.begin() } -> std::same_as<typename T::iterator>;
    { range.end() } -> std::same_as<typename T::iterator>;
    { range.size() } -> std::convertible_to<std::size_t>;
};
```

C++20 Concepts & Constraints

Theory

📌 4. Requires Expressions and Clauses (2/3)

4. Nested Requirements - Check boolean constants:

```
template<typename T>
concept SmallType = requires {
    requires sizeof(T) <= 8;
    requires std::is_trivially_copyable_v<T>;
};
```

Compound Requirement Syntax:

```
{ expression } noexcept -> concept;
//    ^           ^          ^
//    |           |          └── Return type constraint
//    |           └──────────── Optional noexcept check
//    └──────────────────────── Expression to evaluate
```

Examples:

```
template<typename T>
concept Addable = requires(T a, T b) {
    // Check that a + b returns T
    { a + b } -> std::same_as<T>;

    // Check that operation is noexcept
    { a + b } noexcept -> std::same_as<T>;

    // Check convertible (allows implicit conversion)
    { a + b } -> std::convertible_to<T>;
};
```

C++20 Concepts & Constraints

Theory

📌 4. Requires Expressions and Clauses (3/3)

Practical Example - Iterator Concept:

```
template<typename I>
concept ForwardIterator = requires(I it) {
    // Type requirements
    typename std::iterator_traits<I>::value_type;
    typename std::iterator_traits<I>::difference_type;
    typename std::iterator_traits<I>::reference;

    // Simple requirements
    ++it;
    it++;

    // Compound requirements
    { *it } -> std::same_as<typename std::iterator_traits<I>::reference>;
    { it == it } -> std::convertible_to<bool>;
    { it != it } -> std::convertible_to<bool>;

    // Nested requirement
    requires std::copyable<I>;
};
```

C++20 Concepts & Constraints

Theory

📌 5. Constrained Auto and Abbreviated Function Templates (1/3)

Constrained Auto (C++20):

Before C++20, `auto` accepted ANY type. C++20 allows constraining `auto`:

```
// Pre-C++20: Unconstrained
auto x = getValue();  // Could be any type

// C++20: Constrained auto
std::integral auto x = getValue();  // Must be integral type
std::floating_point auto y = getFloat();  // Must be floating-point
std::ranges::range auto container = getRange();  // Must be a range
```

Abbreviated Function Templates:

Function parameters with constrained `auto` create **abbreviated function templates**:

```
// Traditional template
template<std::integral T>
void print(T value) {
    std::cout << value << '\n';
}

// Abbreviated function template (equivalent!)
void print(std::integral auto value) {
    std::cout << value << '\n';
}

// Multiple constrained parameters
void compare(std::integral auto a, std::floating_point auto b) {
    // a is some integral type
    // b is some floating-point type
}

// Deduced return type with constraint
std::integral auto square(std::integral auto x) {
    return x * x;  // Return type deduced and constrained
}
```

C++20 Concepts & Constraints

Theory

📌 5. Constrained Auto and Abbreviated Function Templates (2/3)

Comparison Table:

Traditional Template	Abbreviated Template	Notes
`template<typename T> void f(T x)`	`void f(auto x)`	Unconstrained
`template<std::integral T> void f(T x)`	`void f(std::integral auto x)`	Constrained
`template<typename T, typename U> void f(T a, U b)`	`void f(auto a, auto b)`	Multiple types
`template<std::integral T> T f(T x)`	`std::integral auto f(std::integral auto x)`	Constrained return

When to Use:

Use Case	Prefer
Short functions	Abbreviated template
Complex constraints	Traditional template
Multiple related parameters	Traditional template
Lambdas	Abbreviated template
Library interfaces	Traditional template (more explicit)

C++20 Concepts & Constraints

Theory

📌 5. Constrained Auto and Abbreviated Function Templates (3/3)

Lambda with Constrained Auto:

```
// Pre-C++20: Unconstrained
auto lambda1 = [](auto x) { return x * 2; };

// C++20: Constrained
auto lambda2 = [](std::integral auto x) { return x * 2; };

lambda2(10);      // ✅ OK
lambda2(3.14);    // ❌ Error: double is not integral
```

C++20 Concepts & Constraints

Theory

📌 6. Concept Composition (&&, ||, !) (1/3)

Logical Composition:

Concepts can be composed using logical operators:

```
// AND composition
template<typename T>
concept SignedIntegral = std::integral<T> && std::is_signed_v<T>;

// OR composition
template<typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;

// NOT composition
template<typename T>
concept NotPointer = !std::is_pointer_v<T>;

// Complex composition
template<typename T>
concept ArithmeticNonChar =
    (std::integral<T> || std::floating_point<T>) &&
    !std::same_as<T, char>;
```

Parentheses and Precedence:

```
// Operator precedence: ! > && > ||
template<typename T>
concept A = std::integral<T> || std::floating_point<T> && std::is_signed_v<T>;
// Equivalent to: std::integral<T> || (std::floating_point<T> && std::is_signed_v<T>)

// Use parentheses for clarity:
template<typename T>
concept B = (std::integral<T> || std::floating_point<T>) && std::is_signed_v<T>;
```

C++20 Concepts & Constraints

Theory

📌 6. Concept Composition (&&, ||, !) (2/3)

Practical Examples:

```
// Copyable or movable
template<typename T>
concept CopyableOrMovable = std::copyable<T> || std::movable<T>;

// Container that is iterable and sized
template<typename T>
concept SizedContainer = requires(T container) {
    { container.begin() } -> std::input_or_output_iterator;
    { container.end() } -> std::input_or_output_iterator;
    { container.size() } -> std::convertible_to<std::size_t>;
} && !std::is_array_v<T>;

// Arithmetic but not bool
template<typename T>
concept ArithmeticNotBool = std::is_arithmetic_v<T> && !std::same_as<T, bool>;
```

C++20 Concepts & Constraints

Theory

📌 6. Concept Composition (&&, ||, !) (3/3)

Overloading with Composed Concepts:

```
// Overload for integral types
template<std::integral T>
void process(T value) {
    std::cout << "Integral: " << value << '\n';
}

// Overload for floating-point types
template<std::floating_point T>
void process(T value) {
    std::cout << "Float: " << value << '\n';
}

// Overload for anything else copyable
template<typename T>
    requires std::copyable<T> && !std::integral<T> && !std::floating_point<T>
void process(T value) {
    std::cout << "Other copyable type\n";
}

process(42);        // → Integral: 42
process(3.14);      // → Float: 3.14
process(std::string("hello"));  // → Other copyable type
```

C++20 Concepts & Constraints

Theory

📌 7. Subsumption Rules (1/3)

Subsumption determines which concept is "more constrained" for overload resolution.

The Rule:

Concept A **subsumes** concept B if B's constraints logically imply A's constraints.

Example:

```
template<typename T>
concept Integral = std::is_integral_v<T>;

template<typename T>
concept SignedIntegral = Integral<T> && std::is_signed_v<T>;
//                       ^^^^^^^^^^^ This implies SignedIntegral subsumes Integral

// Overload resolution:
template<Integral T>
void func(T x) { std::cout << "Integral\n"; }

template<SignedIntegral T>
void func(T x) { std::cout << "SignedIntegral\n"; }  // More constrained!

func(42);           // → "SignedIntegral" (int is signed integral)
func(42u);          // → "Integral" (unsigned int is integral but not signed)
```

Why This Works:

```
SignedIntegral = Integral && IsSigned
                 ^^^^^^^^^
                 Contains Integral as part of its constraint

Therefore: SignedIntegral subsumes Integral
```

C++20 Concepts & Constraints

Theory

📌 7. Subsumption Rules (2/3)

Subsumption Chain:

```
template<typename T>
concept A = std::is_arithmetic_v<T>;

template<typename T>
concept B = A<T> && std::is_integral_v<T>;

template<typename T>
concept C = B<T> && std::is_signed_v<T>;

// Subsumption chain: C subsumes B subsumes A
// C is most constrained, A is least constrained

template<A T> void f(T) { std::cout << "A\n"; }
template<B T> void f(T) { std::cout << "B\n"; }
template<C T> void f(T) { std::cout << "C\n"; }

f(3.14);   // → "A" (float: arithmetic but not integral)
f(42u);    // → "B" (unsigned: integral but not signed)
f(42);     // → "C" (int: signed integral - most constrained)
```

Subsumption Comparison Table:

Concepts	Subsumption?	Why?
`A` vs `A && B`	`A && B` subsumes `A`	More constrained
`A` vs `A || B`	Neither	OR doesn't imply subsumption
`A && B` vs `A && B && C`	`A && B && C` subsumes `A && B`	Adding constraints
`A && B` vs `B && A`	Equivalent	Order doesn't matter

C++20 Concepts & Constraints

Theory

📌 7. Subsumption Rules (3/3)

Common Pitfall - OR Does NOT Subsume:

```
template<typename T>
concept A = std::integral<T>;

template<typename T>
concept B = std::floating_point<T>;

template<typename T>
concept C = A<T> || B<T>;

// ❌ AMBIGUOUS - neither subsumes the other!
template<A T> void f(T) { std::cout << "A\n"; }
template<C T> void f(T) { std::cout << "C\n"; }

f(42);  // ❌ Error: ambiguous - int satisfies both A and C, but A doesn't subsume C
```

Fix: Make one subsume the other:

```
template<A T> void f(T) { std::cout << "A\n"; }

// Make this more constrained
template<typename T>
    requires C<T> && !A<T>  // Explicitly exclude A
void f(T) { std::cout << "C but not A\n"; }

f(42);    // → "A"
f(3.14);  // → "C but not A"
```

C++20 Concepts & Constraints

Theory

📌 8. Concepts vs SFINAE - Direct Comparison (1/3)

SFINAE (Pre-C++20) Approach:

```
// ❌ C++17 SFINAE: Verbose and hard to read
template<typename T,
         typename = std::enable_if_t<std::is_arithmetic_v<T>>>
T add(T a, T b) {
    return a + b;
}

// Overload for non-arithmetic (uses different SFINAE trick)
template<typename T,
         typename = std::enable_if_t<!std::is_arithmetic_v<T>>,
         typename = void>  // Extra parameter to avoid redefinition
T add(T a, T b) {
    return a.add(b);  // Call member function
}
```

C++20 Concepts Approach:

```
// ✅ C++20 Concepts: Clear and readable
template<std::arithmetic T>
T add(T a, T b) {
    return a + b;
}

// Overload for non-arithmetic
template<typename T>
    requires (!std::arithmetic<T>) && requires(T a, T b) { a.add(b); }
T add(T a, T b) {
    return a.add(b);
}
```

C++20 Concepts & Constraints

Theory

📌 8. Concepts vs SFINAE - Direct Comparison (2/3)

Comparison Table:

Aspect	SFINAE (C++17)	Concepts (C++20)
Syntax	`std::enable_if_t<condition>`	`template<Concept T>`
Readability	⭐ (cryptic)	⭐⭐⭐⭐⭐ (self-documenting)
Error messages	50-200 lines	5-10 lines
Overload resolution	Complex dummy parameters	Natural subsumption
Composability	Nested enable_if	`&&`, `||`, `!`
Compile time	Slower	Faster
Learning curve	Steep	Gentle

Error Message Comparison:

SFINAE Error (C++17):

```
error: no matching function for call to 'add(std::__cxx11::basic_string<char>, std::__cxx11::basic_string<char>)'
note: candidate: template<class T, class> T add(T, T)
note:   template argument deduction/substitution failed:
note: couldn't deduce template parameter ''
[... 50 more lines of template instantiation backtrace ...]
```

Concepts Error (C++20):

```
error: no matching function for call to 'add(std::string, std::string)'
note: candidate: template<arithmetic T> T add(T, T)
note:   constraints not satisfied
note:   the required type 'std::string' does not satisfy 'arithmetic'
```

C++20 Concepts & Constraints

Theory

📌 8. Concepts vs SFINAE - Direct Comparison (3/3)

Migration Example:

```
// ❌ SFINAE: Check if type has begin() and end()
template<typename T>
auto process(T container)
    -> decltype(container.begin(), container.end(), void())
{
    // Process container
}

// ✅ Concepts: Same check, much clearer
template<typename T>
    requires requires(T c) { c.begin(); c.end(); }
void process(T container) {
    // Process container
}

// ✅ Even better: Use standard concept
template<std::ranges::range T>
void process(T container) {
    // Process container
}
```

Key Takeaway:

- **Use Concepts** for all new C++20 code
- **SFINAE is legacy** - only needed for C++17 compatibility
- **Concepts are strictly better** - no reason to use SFINAE if you have C++20

C++20 Concepts & Constraints

Edge Cases

📌 1. Requires-Clause vs Requires-Expression

The Confusion:

```
// These look similar but are DIFFERENT!

// 1. Requires-clause (constraint on template)
template<typename T>
    requires std::integral<T>
void func1(T x) {}

// 2. Requires-expression (checks validity at compile-time)
template<typename T>
concept HasPushBack = requires(T container) {
    container.push_back(0);
};
```

Key Difference:

- **Requires-clause**: Boolean condition that must be true for template instantiation
- **Requires-expression**: Returns bool checking if code is valid

Nested Example:

```
template<typename T>
concept Complex = requires {
    //            ^^^^^^^^ Requires-expression starts
    requires std::integral<T>;  // Nested requires-clause!
    //^^^^^^^ Requires-clause inside requires-expression
};
```

Rule: `requires requires` is valid C++20 and means:
- Outer `requires`: Start a requires-expression
- Inner `requires`: Check a compile-time boolean condition

C++20 Concepts & Constraints

Edge Cases

📌 2. Concept Specialization is NOT Allowed

```
// ❌ ERROR: Cannot specialize concepts!
template<typename T>
concept Printable = requires(T x) { std::cout << x; };

// This is ILLEGAL:
template<>
concept Printable<MyClass> = true;  // ❌ Error!

// Solution: Use if constexpr or function overloading
```

C++20 Concepts & Constraints

Edge Cases

📌 3. Constraints on Non-Type Template Parameters

```
// ✅ Concepts work on non-type parameters too!
template<std::integral auto N>
    requires (N > 0)  // Constrain the value!
struct Array {
    int data[N];
};

Array<10> arr1;   // ✅ OK
Array<-5> arr2;   // ❌ Error: constraint not satisfied
```

C++20 Concepts & Constraints

Edge Cases

📌 4. Ambiguous Overloads with Equivalent Concepts

```
template<typename T>
concept A = std::integral<T>;

template<typename T>
concept B = std::is_integral_v<T>;  // Logically equivalent to A!

// ❌ AMBIGUOUS:
template<A T> void func(T) { std::cout << "A\n"; }
template<B T> void func(T) { std::cout << "B\n"; }

func(42);  // ❌ Error: ambiguous (compiler can't tell A and B apart)
```

Why: Compiler doesn't analyze logical equivalence, only syntactic subsumption.

C++20 Concepts & Constraints

Edge Cases

📌 5. Requires in Member Functions

```
template<typename T>
struct Container {
    // ✅ Can constrain member functions
    void push(T value) requires std::copyable<T> {
        // Only available if T is copyable
    }

    // Different overload for movable-only types
    void push(T value) requires std::movable<T> && !std::copyable<T> {
        // Only available if T is movable but not copyable
    }
};

Container<std::unique_ptr<int>> c;
c.push(std::make_unique<int>(42));  // Calls movable-only version
```

C++20 Concepts & Constraints

Edge Cases

📌 6. Concepts Don't Short-Circuit

```
template<typename T>
concept HasSize = requires(T x) {
    x.size();  // Check if size() exists
};

template<typename T>
concept LargeContainer = HasSize<T> && (T{}.size() > 100);
//                                      ^^^^^^^^^ This evaluates even if HasSize is false!

// Problem: If T doesn't have size(), compilation fails trying to evaluate T{}.size()

// ✅ Fix: Use requires for runtime checks
template<typename T>
concept LargeContainer = requires(T x) {
    x.size();
    requires (sizeof(typename T::value_type) * 100 < 1000);  // Compile-time only
};
```

C++20 Concepts & Constraints

Edge Cases

📌 7. Dependent Names in Concepts

```
template<typename T>
concept HasValueType = requires {
    typename T::value_type;  // Check if value_type exists
};

// ❌ This doesn't work as expected:
template<typename T>
concept IntContainer = HasValueType<T> &&
                       std::same_as<T::value_type, int>;  // ❌ Error: dependent name

// ✅ Fix: Use typename
template<typename T>
concept IntContainer = HasValueType<T> &&
                       std::same_as<typename T::value_type, int>;  // ✅ OK
```

C++20 Concepts & Constraints

Code Examples

📌 1. Type-Safe Printf with Concepts

Output:

```
#include <iostream>
#include <concepts>
#include <string>

// Concept: Types that can be printed to cout
template<typename T>
concept Printable = requires(T value) {
    { std::cout << value } -> std::same_as<std::ostream&>;
};

// Base case: no arguments
void print() {
    std::cout << '\n';
}

// Recursive case: print first argument, then rest
template<Printable First, Printable... Rest>
void print(First first, Rest... rest) {
    std::cout << first;
    if constexpr (sizeof...(Rest) > 0) {
        std::cout << ' ';
        print(rest...);
    } else {
        std::cout << '\n';
    }
}

// Test:
int main() {
    print(1, 2, 3);                          // ✅ "1 2 3"
    print("Hello", std::string("World"));    // ✅ "Hello World"
    print(3.14, "pi", 42);                   // ✅ "3.14 pi 42"

    // print(std::vector<int>{});            // ❌ Error: vector not Printable
}
```

```
['1 2 3\nHello World\n3.14 pi 42']
```

C++20 Concepts & Constraints

Code Examples

📌 2. Generic Algorithm with Multiple Constraints

```
#include <concepts>
#include <vector>
#include <algorithm>

// Find and return index of element in range
template<std::ranges::range R, typename T>
    requires std::equality_comparable_with<std::ranges::range_value_t<R>, T>
std::optional<size_t> find_index(const R& range, const T& value) {
    size_t index = 0;
    for (const auto& elem : range) {
        if (elem == value) {
            return index;
        }
        ++index;
    }
    return std::nullopt;
}

// Test:
int main() {
    std::vector<int> nums = {10, 20, 30, 40};

    auto idx = find_index(nums, 30);
    if (idx) {
        std::cout << "Found at index: " << *idx << '\n';  // → 2
    }

    // Works with different container types:
    std::array<std::string, 3> words = {"hello", "world", "foo"};
    auto idx2 = find_index(words, std::string("world"));
    if (idx2) {
        std::cout << "Found at index: " << *idx2 << '\n';  // → 1
    }
}
```

C++20 Concepts & Constraints

Code Examples

📌 3. Smart Pointer Concept and Generic Deleter

```
#include <concepts>
#include <memory>

// Concept: Something that acts like a pointer
template<typename T>
concept Pointer = requires(T ptr) {
    *ptr;                    // Dereferenceable
    ptr == nullptr;          // Comparable to nullptr
    { ptr.get() } -> std::convertible_to<typename T::element_type*>;
};

// Generic function that works with any smart pointer
template<Pointer P>
void process(P ptr) {
    if (ptr != nullptr) {
        std::cout << "Processing: " << *ptr << '\n';
    }
}

int main() {
    auto unique = std::make_unique<int>(42);
    auto shared = std::make_shared<int>(100);

    process(unique);  // ✅ Works with unique_ptr
    process(shared);  // ✅ Works with shared_ptr
}
```

C++20 Concepts & Constraints

Code Examples

📌 4. Arithmetic Operations with Mixed Types

```
#include <concepts>

// Concept: Two types that can be added
template<typename T, typename U>
concept Addable = requires(T a, U b) {
    { a + b };
};

// Generic add that works with any arithmetic types
template<typename T, typename U>
    requires (std::arithmetic<T> && std::arithmetic<U>)
auto add(T a, U b) {
    return a + b;  // Return type deduced (int + double = double)
}

// Specialization for integral types (faster path)
template<std::integral T, std::integral U>
auto add(T a, U b) {
    return static_cast<std::common_type_t<T, U>>(a) + b;
}

int main() {
    std::cout << add(5, 10) << '\n';        // → 15 (integral specialization)
    std::cout << add(3.14, 2.71) << '\n';   // → 5.85 (arithmetic version)
    std::cout << add(5, 3.14) << '\n';      // → 8.14 (mixed types)
}
```

C++20 Concepts & Constraints

Code Examples

📌 5. Container Adapter with Concept Constraints

```
#include <concepts>
#include <vector>
#include <deque>
#include <list>

// Concept: Container that supports push_back
template<typename C>
concept BackInsertable = requires(C container, typename C::value_type value) {
    container.push_back(value);
    { container.size() } -> std::convertible_to<size_t>;
};

// Generic container wrapper
template<BackInsertable Container>
class Stack {
    Container data;
public:
    void push(const typename Container::value_type& value) {
        data.push_back(value);
    }

    typename Container::value_type pop() {
        auto value = data.back();
        data.pop_back();
        return value;
    }

    size_t size() const { return data.size(); }
};

int main() {
    Stack<std::vector<int>> stack1;        // ✅ OK
    Stack<std::deque<int>> stack2;         // ✅ OK
    // Stack<std::list<int>> stack3;       // ❌ Error: list not BackInsertable (no push_back indexing)
}
```

C++20 Concepts & Constraints

Code Examples

📌 6. Compile-Time Validation with Concepts

```
#include <concepts>

// Concept: Type must be small and trivial
template<typename T>
concept SmallPOD = std::is_trivially_copyable_v<T> && (sizeof(T) <= 16);

// Efficient pass-by-value for small types
template<SmallPOD T>
void process(T value) {
    std::cout << "Pass by value (small POD)\n";
    // Process directly
}

// Pass-by-reference for large types
template<typename T>
    requires (!SmallPOD<T> && std::copyable<T>)
void process(const T& value) {
    std::cout << "Pass by reference (large type)\n";
    // Process by reference
}

struct SmallStruct { int x; };           // 4 bytes
struct LargeStruct { int arr[100]; };    // 400 bytes

int main() {
    SmallStruct small;
    LargeStruct large;

    process(small);   // → "Pass by value (small POD)"
    process(large);   // → "Pass by reference (large type)"
}
```

C++20 Concepts & Constraints

Code Examples

📌 7. Iterator Range Concept

```
#include <concepts>
#include <iterator>
#include <vector>

// Concept: A pair of iterators forming a valid range
template<typename I>
concept Iterator = requires(I it) {
    ++it;
    *it;
    it != it;
};

template<typename I>
concept IteratorPair = Iterator<I> && requires(I begin, I end) {
    { begin != end } -> std::convertible_to<bool>;
};

// Generic sum function for any iterator range
template<std::input_iterator I>
auto sum_range(I begin, I end) {
    using ValueType = typename std::iterator_traits<I>::value_type;
    ValueType total{};
    for (auto it = begin; it != end; ++it) {
        total += *it;
    }
    return total;
}

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};
    auto result = sum_range(nums.begin(), nums.end());
    std::cout << "Sum: " << result << '\n';  // → 15
}
```

C++20 Concepts & Constraints

Code Examples

📌 8. Concept-Based Factory Pattern (1/2)

```
#include <concepts>
#include <memory>
#include <map>
#include <string>

// Concept: Types that can be default-constructed
template<typename T>
concept DefaultConstructible = std::default_initializable<T>;

// Base interface
struct Shape {
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

// Derived shapes
struct Circle : Shape {
    void draw() const override { std::cout << "Drawing Circle\n"; }
};

struct Square : Shape {
    void draw() const override { std::cout << "Drawing Square\n"; }
};

// Generic factory
template<std::derived_from<Shape> T>
    requires DefaultConstructible<T>
class ShapeFactory {
public:
    static std::unique_ptr<Shape> create() {
        return std::make_unique<T>();
    }
};

int main() {
    auto circle = ShapeFactory<Circle>::create();
    auto square = ShapeFactory<Square>::create();

    circle->draw();  // → "Drawing Circle"
// ...continues on next slide
```

📌 8. Concept-Based Factory Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    square->draw();  // → "Drawing Square"
}
```

C++20 Concepts & Constraints

Code Examples

📌 9. Mathematical Vector with Concepts (1/2)

```
#include <concepts>
#include <array>
#include <cmath>

// Concept: Numeric types suitable for vector operations
template<typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;

template<Numeric T, size_t N>
class Vector {
    std::array<T, N> data;

public:
    Vector() = default;

    template<typename... Args>
        requires (sizeof...(Args) == N)
    Vector(Args... args) : data{static_cast<T>(args)...} {}

    T& operator[](size_t i) { return data[i]; }
    const T& operator[](size_t i) const { return data[i]; }

    // Vector addition
    Vector operator+(const Vector& other) const {
        Vector result;
        for (size_t i = 0; i < N; ++i) {
            result[i] = data[i] + other[i];
        }
        return result;
    }

    // Dot product
    T dot(const Vector& other) const {
        T sum = 0;
        for (size_t i = 0; i < N; ++i) {
            sum += data[i] * other[i];
        }
        return sum;
    }
// ...continues on next slide
```

C++20 Concepts & Constraints

Code Examples

📌 9. Mathematical Vector with Concepts (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    // Magnitude (only for floating-point)
    auto magnitude() const requires std::floating_point<T> {
        T sum_sq = 0;
        for (size_t i = 0; i < N; ++i) {
            sum_sq += data[i] * data[i];
        }
        return std::sqrt(sum_sq);
    }
};

int main() {
    Vector<int, 3> v1(1, 2, 3);
    Vector<int, 3> v2(4, 5, 6);

    auto v3 = v1 + v2;  // → (5, 7, 9)
    std::cout << "Dot product: " << v1.dot(v2) << '\n';  // → 32

    Vector<double, 3> v4(3.0, 4.0, 0.0);
    std::cout << "Magnitude: " << v4.magnitude() << '\n';  // → 5.0
}
```

C++20 Concepts & Constraints

Code Examples

📌 10. Concept-Constrained Visitor Pattern

```
#include <concepts>
#include <variant>
#include <string>

// Concept: Callable that can visit all variant alternatives
template<typename F, typename... Types>
concept VariantVisitor = (std::invocable<F, Types> && ...);

// Generic visit wrapper with concept check
template<typename... Types, VariantVisitor<Types...> F>
auto visit_variant(std::variant<Types...>& var, F&& visitor) {
    return std::visit(std::forward<F>(visitor), var);
}

int main() {
    std::variant<int, double, std::string> value;

    // Lambda that handles all types
    auto printer = [](auto&& arg) {
        std::cout << "Value: " << arg << '\n';
    };

    value = 42;
    visit_variant(value, printer);  // → "Value: 42"

    value = 3.14;
    visit_variant(value, printer);  // → "Value: 3.14"

    value = std::string("Hello");
    visit_variant(value, printer);  // → "Value: Hello"
}
```

What You Learned

Key Takeaways

- ✓ The Problem They Solve
- ✓ Concept Syntax and Declaration
- ✓ Standard Concepts Library (``)
- ✓ Requires Expressions and Clauses

→ Swipe to learn →